LING285-L3

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 3 - Dynamical Systems I
Date: 9/1/2026
Professor: Khalil Iskarous

Section 1: What a Dynamical System Is

- A **dynamical system** is a system whose state changes over time according to rules. We study dynamical systems in two places: in **animal behavior**, such as an octopus defending its home or a person speaking, and in **AI learning**, where a computer program changes its internal numbers as it is trained.
- Every dynamical system can be described with three parts: **1) Elements, 2) Relations, 3) Goal.** The **elements** are the pieces that make up the system. The **relations** are the ways those pieces are connected and coordinated with each other. The **goal** is what the whole coordinated system is working to achieve.
    - Using these three parts lets us describe very different systems in the same terms, whether the system is an animal's body, a vocal tract, or a machine learning program.

Section 2: The Octopus as a Dynamical System

- An octopus defending its den is a clear example of a dynamical system in behavior. Its **elements** are its **8 arms** and the **ground** it is sitting on and pushing against.
- The **relations** in this system are the coordinated connections between each individual arm, the ground, and the fish the octopus is interacting with. These relations are **planned by a neural system** that lives in the octopus's head and also throughout the rest of its body. The octopus does not control its arms from one central point alone; the planning is spread across the body.
- The **goal** of this system is to **keep the den.** The octopus coordinates its arms, its grip on the ground, and the movement of the fish so that it can protect its home.

Section 3: The Speech System as a Dynamical System

- Human speech is also a dynamical system, and it has many more elements than the octopus example. The **elements** are the **many parts of the tongue**, the **lips**, the **velum**, the **larynx**, the **palate**, and the **pharyngeal wall**.
    - The **tongue** is not one single element; its tip, body, and root can each move somewhat independently, so it counts as many parts.
    - The **velum** is the soft back part of the roof of the mouth. It raises to close off the nasal passage or lowers to let air flow through the nose.
    - The **larynx** is the voice box in the throat, which contains the vocal folds that vibrate to produce voicing.
    - The **palate** is the hard roof of the mouth, and the **pharyngeal wall** is the back wall of the throat behind the tongue.
- The **relations** between all these parts are **planned by the brain**. Producing a single word requires the lips, tongue, velum, and larynx to move at precisely coordinated times relative to each other.
- The **goal** of the speech system is to **convey a thought**. All of this coordinated movement exists so that an idea in one person's mind can be transmitted to another person.
- A **real-time MRI** recording of a speaker shows this system in action. The image is a side view of the head cut down the middle, showing the lips at the front, the tongue filling the mouth, the palate and velum above the tongue, and the throat and spine running down the back. Over about three seconds of speech, the lips, jaw, and tongue change shape and position continuously, which shows that speech is a set of many parts moving together over time rather than a series of frozen positions.

Section 4: Dynamical Systems Throughout the Body

- All of our **daily voluntary actions**, such as walking or reaching, are dynamical systems. The **involuntary actions** of the **heart**, the **pancreas**, and **neurons** are dynamical systems too, since each involves parts whose activity changes over time in coordinated ways.
- The elements of a dynamical system can be very large or very small. They can be as big as **organs** or parts of organs, and as small as **cells** and the **chemicals** inside cells. **Biology, chemistry, and physics** are all fields that study dynamical systems at these different scales.
- The **speech system** is a **very complex dynamical system**, since it combines many body parts, many muscles, and brain planning all working toward one goal.
- There are so many dynamical systems in nature that we can **use one dynamical system to study another**. Two systems can be extremely different on the surface, such as an octopus arm and a human tongue, while still sharing an **underlying similarity** in how their elements, relations, and goals are organized.
    - This idea is described as **going up Diotima's ladder**. Diotima is a figure in Plato's *Symposium* who describes rising from appreciating particular individual examples to understanding the general form they all share. Studying many different dynamical systems moves us from individual cases toward the general principles that all of them have in common.

Section 5: The Octopus Arm "Slap" Study

- The octopus example comes from a published study: **"A syntactic analysis of a complex motor action: the octopus arm 'slap'"** by **Jean Alupay, Jennifer Mather, and Khalil Iskarous**, published in **Marine Biology in 2023**. The paper was received on 15 February 2023 and accepted on 1 June 2023.
- The researchers studied an octopus of the genus ***Abdopus***, filmed in **tidal pools in Okinawa, Japan**, as it **"slapped" at fish** in the wild. A **slap** is a quick arm strike the octopus uses against a fish near its den.
    - In total, **19 slaps were analyzed**.
    - The arm movements were measured in two **reference frames**. An **external reference frame** measures position relative to the surrounding environment, and an **animal-centered reference frame** measures position relative to the octopus's own body.
- The octopus built each slap out of two **kinematic primitives**, which are basic building-block movements. These two primitives are **rotation** and **translation** of its **hydrostatic arms**. Rotation means turning the arm around a point, and translation means moving the arm from one place to another without turning it.
    - A **muscular hydrostat** is a body structure with no bones that is supported by muscles pushing against each other around an incompressible core. The octopus arm is a muscular hydrostat, which is why it can bend at any point along its length.
    - The octopus combined these primitives **serially**, meaning one after another, and **in parallel**, meaning at the same time.
- By combining a small number of primitives, the octopus keeps its movement **flexible** while only having to control **a few degrees of freedom**. A **degree of freedom** is one independent way a system can move. An octopus arm could in principle bend in almost unlimited ways, so limiting control to a few combined primitives makes the movement manageable. The authors named this concept **flexible rigidity**.
- The slap supports **Flash and Hochner's embodied organization** view of motor behavior. This view holds that much of the organization of movement comes from the structure of the body itself, not only from commands in the brain. It also supports their idea that **motor primitives can combine syntactically** to form a complex action.
    - **Syntax** in language is the set of rules for combining words into larger structures like sentences. Calling the slap **syntactic** means that small movement units are combined by rules into a larger action, in the same way words are combined into sentences. This is a direct case of using one dynamical system, the octopus arm, to study another, human language.
- The octopus uses **sensory feedback** about the position of the moving fish together with **feed-forward** motor primitives. **Feedback** is information coming in from the senses during the movement that allows correction. **Feed-forward** control means sending out a

pre-planned movement without waiting for correction. Combining the two lets the octopus build complex actions that stay in **dynamic equilibrium** with the environment, meaning the octopus keeps adjusting so that its actions remain balanced with a changing situation.

- These findings give a more realistic picture of how a complex behavior allows an animal to **coordinate with its environment**. The paper's keywords are **Octopus, Muscular hydrostat, Motor control,** and ***Abdopus***.
- The book ***Octopus: The Ocean's Intelligent Invertebrate — A Natural History*** by **Jennifer A. Mather, Roland C. Anderson, and James B. Wood** is a broader source on octopus behavior and intelligence. Jennifer Mather is also a co-author of the slap study.

Section 6: AI as a Model of Animal Psychological Skills

- **AI** is a **model of several animal psychological skills**. It is a system built to imitate abilities that animals, including humans, already have. The five skills are **knowing, recognizing, attending, acting,** and **learning**.
  - **Knowing** is holding stored information about the world, such as information about how words are used.
  - **Recognizing** is identifying what something is when it is encountered, such as recognizing a word or a face.
  - **Attending** is focusing on the parts of the input that matter most at a given moment.
  - **Acting** is producing an output, such as a movement for an animal or a word for a language model.
  - **Learning** is changing internal knowledge based on experience so that future performance improves.
- The **Cloze Procedure** is an early example of measuring one of these skills in humans. It was introduced by **Wilson L. Taylor** in **"'Cloze Procedure': A New Tool for Measuring Readability,"** published in ***Journalism Quarterly*** in **Fall 1953**.
  - In a **cloze task**, words are removed from a passage and readers must fill in the blanks. The more easily readers can guess the missing words, the more readable and predictable the text is.
  - Taylor's article was the first full statement of the method and its theory, which had first been introduced briefly at a workshop at the **1953 AEJ convention**. It included findings from **three pilot studies** and **two experiments** comparing cloze results with those of **two readability formulas**.
  - Filling in a missing word from its context is the same basic skill that modern language AI is built to perform: predicting a word from the words around it.

Section 7: How GPT Predicts the Next Word

- In **GPT**, each word is represented as a **vector**, which is a list of **thousands of numbers**. We call these vectors **x's** or **tokens**. In the example sentence "**chickens cackle and ducks**," each of the four words is its own tall column of numbers, one column for each token.
- All of the AI's **knowledge** about speech, language, or anything else is stored in **a few trillion numbers** called **weights**, or **w's**. The weights are not a dictionary or a list of facts; they are numbers whose values together determine how the system responds to input.
- The AI combines its knowledge, the **W's**, with the input vectors, the **x's**, using mostly the four basic operations: **addition, subtraction, multiplication, and division** ($+, -, \times, \div$). The complexity of the system comes from the enormous number of these simple calculations, not from any complicated single operation.
- These calculations produce a **probability distribution function** over possible next words. A **probability distribution** assigns each possible word a probability, and all the probabilities together add up to one. This distribution is the system's way of saying, "this is my guess for the next word."
  - Given "chickens cackle and ducks," the distribution is spread across a line of possible words that includes "**a**," "**cat**," "**quack**," and "**Zulu**." The curve over this line has small bumps over many words, showing that many words get a little probability, and a tallest peak over "**quack**," showing that "quack" is the system's most likely guess.

- The way the **W's combine with the X's** to produce this prediction is itself a **dynamical system**. **Learning** is a simpler dynamical system than this prediction process, which makes it the easier one to work through first.

Section 8: Where the Weights Come From

- The weights, which hold all of the system's knowledge, come from a training process that starts with no knowledge at all. The approach is summed up as **"Fake it till you make it!"**
- Training follows a repeated sequence of steps:
  1. The system **starts with random W's**. These are completely random numbers, and any values at all can be used as the starting point.
  2. The system is given a sequence of words, such as "**chickens cackle and ducks**," and is asked to **predict the next word** by combining its W's with the input vectors.
  3. With random weights, the prediction is poor. The system's distribution might have its tallest peak over "**cat**," with only low, uneven probability spread across the other words, including "quack."
  4. We already know the correct next word is "**quack**." The **target distribution** puts all of the probability on "quack" as a single sharp spike and zero probability on every other word. We tell the system to **change its W's** so that its output distribution looks more like this target.
  5. The system is exposed to **more sequences** and asked to predict the next word again, **billions of times**, and each time its W's are changed to **improve its guessing**.
- This repeated cycle of guessing, comparing to the correct answer, and adjusting the weights is **how LLMs learn and know**. An **LLM**, or **large language model**, gets all of its knowledge from this process of gradually correcting random weights over billions of examples.

Section 9: Two Tricks of AI Learning

- AI learning depends on **two tricks**, each captured by a phrase in an image generated by **ChatGPT**.
- The first trick is **"Fake It Till You Make It."** The system begins with **random weights** and makes predictions right away, even though those first predictions are essentially meaningless. It does not wait until it has knowledge before acting; it acts first and gains knowledge along the way.
- The second trick is **"Learn by Making Mistakes."** Each wrong prediction produces an **error**, which is the difference between what the system guessed and what was correct. The system uses that error to adjust its weights, so every mistake becomes information that improves the next prediction.

Section 10: Averaging as a Learning Problem

- Finding an **average** seems very easy when we have a short list of numbers. The average of 10, 12, and 14 is found by adding the numbers and dividing by how many there are:

$$\frac{10 + 12 + 14}{3} = 12$$

- This simple formula **does not work for more complex things** like **speech, language, biological molecules, and scenes**, which are exactly the kinds of things AI works with. For those, there is no short list of numbers we can simply add up and divide, so we need a method that improves an estimate step by step instead.
- The averaging method used here works by **learning from the data one piece at a time**:
  - We start with a **list of numbers**.
  - We make a **wild guess**, called x, about what their average is.
  - We improve x through **exposure to the data.** We take one number from the list and examine it with respect to the current value of x, checking how far apart they are.

    - We then improve x by moving it **a bit closer** to the number we observed from the data. This is called **observation-based error learning**, since each observation reveals an error in the current guess and the guess is corrected in the direction of that observation.
    - We **repeat** this with another number, and then another, so that the guess keeps being corrected.
- This is the same logic as training an LLM, applied to a much simpler problem. The random starting guess for x plays the role of the random starting weights, and moving x toward each observed number plays the role of changing the weights toward the correct next word.

[CONTINUE]

Section 11: Starting With a Wild Guess

- **SGD**, or **stochastic gradient descent**, is the learning method that estimates an average by learning from the data one observation at a time. The question it answers is: what is the average, the **essence**, of this list of numbers?

$$[78,\ 67,\ 98,\ 85,\ 76,\ 68,\ 98,\ 72]$$

- The true average of this list is found by adding the eight numbers and dividing by eight. This value serves as the target that the learning process should approach.

$$\frac{78 + 67 + 98 + 85 + 76 + 68 + 98 + 72}{8} = \frac{642}{8} = 80.25$$

- SGD does not start by calculating this average. It starts by picking **any random number** x as the estimate. At **time 0**, the starting estimate is set to 130, which is far above every number in the list.

$$x[0] = 130$$

- The number in brackets is the **time step**, so $x[0]$ is the estimate at time 0, $x[1]$ is the estimate after one update, and so on.
- The **learning rate**, written γ, is set to 25%. The learning rate controls how large each correction is, meaning what fraction of the error the estimate moves on each step.

$$\gamma = 25\% = 0.25$$

- Plotted against the update number, the estimate at time 0 is a single point at 130, while the true average of 80.25 sits far below it as a flat horizontal line.

Section 12: One Observation Creates Error

- The first step of learning is to **randomly pick one number** d from the data. The first observation drawn is 98.

$$d[0] = 98$$

- The current estimate is $x[0] = 130$. The **discrepancy**, also called the **loss**, is the difference between the current estimate and the observation. The loss measures how wrong the estimate is compared to this piece of data.

$$130 - 98 = 32$$

- The discrepancy is positive, which means the estimate is 32 units too high compared to this observation. The average of a data set should be **close to the data**, so an estimate that is far from an observed value is a sign that the estimate needs to change.
- Plotted together at update 0, the estimate is at 130 and the observation is at 98, below it, with the true average of 80.25 lower still.

Section 13: Moving 25% of the Error

- The correction step **subtracts 25% of the discrepancy** from the current estimate. With a discrepancy of 32 and a learning rate of 0.25, the size of the correction is found first.

$$0.25 \times 32 = 8$$

- That correction is subtracted from the current estimate to give the new estimate.

$$x[1] = 130 - 0.25(32)$$

$$x[1] = 130 - 8 = 122$$

- The estimate moves **toward this observation**. It does not jump all the way to 98; it only moves one quarter of the distance, from 130 to 122. Moving only part of the way keeps any single observation from taking over the estimate.
- Plotted over updates, the estimate line drops from 130 at update 0 to 122 at update 1, heading down toward the average line at 80.25.

Section 14: Repeating the Correction

- The same procedure is repeated with a new random observation each time: draw a number, compute the discrepancy, and subtract 25% of that discrepancy from the estimate.
- On the second update, the random observation is 85. The discrepancy between the current estimate of 122 and this observation is computed first.

$$d[1] = 85$$

$$122 - 85 = 37$$

- The correction is 25% of 37, and subtracting it gives the new estimate.

$$0.25 \times 37 = 9.25$$

$$x[2] = 122 - 0.25(37) = 122 - 9.25 = 112.75$$

- The estimate moves toward this observation, dropping from 122 to 112.75.
- On the third update, the random observation is 76. The discrepancy is again positive, since the estimate is still above the observation.

$$d[2] = 76$$

$$112.75 - 76 = 36.75$$

- The correction is 25% of 36.75, which is then subtracted from the estimate.

$$0.25 \times 36.75 = 9.1875$$

$$x[3] = 112.75 - 0.25(36.75) = 112.75 - 9.1875 \approx 103.563$$

- The estimate moves toward this observation, dropping from 112.75 to about 103.563.
- On the fourth update, the random observation is 68.

$$d[3] = 68$$

$$103.563 - 68 = 35.563$$

- The correction is 25% of 35.563, which is subtracted from the estimate.

$$0.25 \times 35.563 \approx 8.891$$

$$x[4] = 103.563 - 0.25(35.563) = 103.563 - 8.891 = 94.672$$

- The estimate moves toward this observation, dropping from 103.563 to 94.672.
- Across the first four updates, the estimate falls steadily: 130, 122, 112.75, 103.563, and 94.672. Every observation so far has been below the estimate, so every correction has pushed the estimate down toward the average line at 80.25.

Section 15: A High Datum Pulls x Upward

- On the fifth update, the random observation is 98, which is **higher** than the current estimate of 94.672. A **datum** is a single data point.

$$d[4] = 98$$

- The discrepancy is now **negative**, since the estimate is below the observation.

$$94.672 - 98 = -3.328$$

- The rule still says to subtract 25% of the discrepancy. The correction is a negative number.

$$0.25 \times (-3.328) = -0.832$$

$$x[5] = 94.672 - 0.25(-3.328) = 94.672 + 0.832 = 95.504$$

- **Subtracting a negative number is the same as adding**, so a negative discrepancy makes the subtraction **move x upward**. The estimate rises slightly from 94.672 to 95.504, toward the high observation.
    - The same single rule handles both cases. When the estimate is above an observation, the discrepancy is positive and the estimate goes down; when the estimate is below an observation, the discrepancy is negative and the estimate goes up. In both cases the estimate moves toward the observation.
- Plotted over updates, the estimate line bends slightly upward at update 5 after four straight drops, which is the first sign that the path does not move smoothly in one direction.

Section 16: Repeating the Correction After the Rise

- On the sixth update, the random observation is 72, which is below the estimate again.

$$d[5] = 72$$

$$95.504 - 72 = 23.504$$

- The correction is 25% of 23.504, which is subtracted from the estimate.

$$0.25 \times 23.504 = 5.876$$

$$x[6] = 95.504 - 0.25(23.504) = 95.504 - 5.876 = 89.628$$

- The estimate moves toward this observation, dropping from 95.504 to 89.628.
- On the seventh update, the random observation is 78.

$$d[6] = 78$$

$$89.628 - 78 = 11.628$$

- The discrepancy is smaller than before, since the estimate is now much closer to the data. The correction is 25% of 11.628.

$$0.25 \times 11.628 = 2.907$$

$$x[7] = 89.628 - 0.25(11.628) = 89.628 - 2.907 = 86.721$$

- The estimate moves toward this observation, dropping from 89.628 to 86.721.
- On the eighth update, the random observation is 67, the lowest number in the list.

$$d[7] = 67$$

$$86.721 - 67 = 19.721$$

- The correction is 25% of 19.721, which is subtracted from the estimate.

$$0.25 \times 19.721 \approx 4.930$$

$$x[8] = 86.721 - 0.25(19.721) = 86.721 - 4.930 = 81.791$$

- The estimate moves toward this observation, dropping from 86.721 to 81.791. After eight updates, the estimate is only about 1.5 above the true average of 80.25.

Section 17: A High Datum Pulls x Upward Again

- On the ninth update, the random observation is 98 once more. The value 98 appears twice in the list, and since each draw is random, the same value can be picked more than once. This is the third time 98 has been drawn.

$$d[8] = 98$$

- The estimate of 81.791 is well below 98, so the discrepancy is negative and fairly large.

$$81.791 - 98 = -16.209$$

- Subtracting 25% of a negative discrepancy moves the estimate upward.

$$0.25 \times (-16.209) \approx -4.052$$

$$x[9] = 81.791 - 0.25(-16.209) = 81.791 + 4.052 = 85.843$$

- A negative discrepancy makes the subtraction move x upward, from 81.791 to 85.843. A single high observation pushed the estimate away from the true average, even though the estimate had been getting very close. This shows that each step responds only to the one observation drawn, not to the whole data set.

Section 18: Final Corrections Toward the Average

- On the tenth update, the random observation is 85, which is extremely close to the current estimate.

$$d[9] = 85$$

$$85.843 - 85 = 0.843$$

- The discrepancy is less than 1, so the correction is tiny.

$$0.25 \times 0.843 \approx 0.211$$

$$x[10] = 85.843 - 0.25(0.843) = 85.843 - 0.211 = 85.632$$

- The estimate barely moves, from 85.843 to 85.632. When an observation agrees closely with the estimate, the estimate hardly changes.
- On the eleventh update, the random observation is 76.

$$d[10] = 76$$

$$85.632 - 76 = 9.632$$

- The correction is 25% of 9.632, which is subtracted from the estimate.

$$0.25 \times 9.632 = 2.408$$

$$x[11] = 85.632 - 0.25(9.632) = 85.632 - 2.408 = 83.224$$

- The estimate moves toward this observation, dropping from 85.632 to 83.224.
- On the twelfth update, the random observation is 68.

$$d[11] = 68$$

$$83.224 - 68 = 15.224$$

- The correction is 25% of 15.224, which is subtracted from the estimate.

$$0.25 \times 15.224 = 3.806$$

$$x[12] = 83.224 - 0.25(15.224) = 83.224 - 3.806 = 79.418$$

- The estimate moves toward this observation, dropping from 83.224 to 79.418. This is less than 1 below the true average of 80.25, even though the process started at 130.
- Over all twelve updates, the estimate went 130, 122, 112.75, 103.563, 94.672, 95.504, 89.628, 86.721, 81.791, 85.843, 85.632, 83.224, and 79.418. The path falls steeply at first, bumps upward twice when 98 is drawn, and ends up hovering near the average line.

Section 19: What SGD Is

- **SGD** consists of two steps. First, **make an initial estimate or guess**, which can be completely random. Second, **use data to improve the estimate**, one observation at a time, by moving the estimate part of the way toward each observation.
- Each word in the name describes part of the method.
  - **Stochastic** means **random observation**. At each step, the data point used for the correction is picked at random from the data set, which is why the estimate is sometimes pushed down and sometimes pulled up.
  - **Gradient** means a **loss-related direction**. The discrepancy tells us both how wrong the estimate is and which direction it needs to move: a positive discrepancy means move down, and a negative discrepancy means move up.
  - **Descent** means **reducing the mistake**. Every correction is chosen so that the error for the observed data point gets smaller.
- **The path jitters, but its tendency is toward the average.** Individual steps can move away from the true average, as happened when 98 was drawn, but over many updates the random pushes up and down balance out and the estimate settles near about 80, the average of the data.
- This is observation-based error learning in its simplest form. The same method, applied to trillions of weights instead of a single number x, is how LLMs change their W's to predict the next word.

Section 20: As the Estimate Decreases, the Loss Decreases

- A **second data set** makes the relationship between the estimate and the loss easier to track.

$$[62,\ 70,\ 78,\ 82,\ 90,\ 98]$$

- The average of this data set is 80.

$$\frac{62 + 70 + 78 + 82 + 90 + 98}{6} = \frac{480}{6} = 80$$

- The **mean motion of SGD** describes how the estimate moves on average. Each random observation is sometimes above and sometimes below the average, so on average the estimate is pulled toward the mean of the data, 80. Replacing the random observation with the average gives the rule for the mean motion.

$$x[t+1] = x[t] - 0.25\,(x[t] - 80)$$

- Here t stands for any time step, so this one equation describes every update. The new estimate $x[t+1]$ equals the old estimate $x[t]$ minus 25% of the distance between the old estimate and 80.
- The **loss** at each time step is defined as one half of the discrepancy squared.

$$L[t] = \tfrac{1}{2}\,(x[t] - 80)^2$$

- **Squaring** the discrepancy makes the loss positive whether the estimate is too high or too low, so both kinds of mistakes count as error. The loss is zero only when the estimate equals 80 exactly.
- The rate at which this loss changes as x changes is equal to the discrepancy $x[t] - 80$. This is why the discrepancy is the **gradient**, the loss-related direction, and why subtracting a fraction of it moves the estimate in the direction that lowers the loss.
- The starting point is set far above the average.

$$x[0] = 140$$

- At time 0, the loss is computed from the discrepancy of 60.

$$L[0] = \tfrac{1}{2}(140 - 80)^2 = \tfrac{1}{2}(60)^2 = \tfrac{1}{2}(3600) = 1800$$

- On the first update, the estimate moves 25% of the way toward 80.

$$x[1] = 140 - 0.25(140 - 80) = 140 - 0.25(60) = 140 - 15 = 125$$

$$L[1] = \tfrac{1}{2}(125 - 80)^2 = \tfrac{1}{2}(45)^2 = \tfrac{1}{2}(2025) = 1012.5$$

- On the second update, the same rule is applied again.

$$x[2] = 125 - 0.25(125 - 80) = 125 - 0.25(45) = 125 - 11.25 = 113.75$$

$$L[2] = \tfrac{1}{2}(113.75 - 80)^2 = \tfrac{1}{2}(33.75)^2 = \tfrac{1}{2}(1139.0625) \approx 569.5$$

- Each update keeps 75% of the previous discrepancy, so the discrepancy shrinks from 60 to 45 to 33.75 and continues shrinking. The estimate curve falls quickly at first and then flattens out as it approaches the target line at 80, reaching about 82 by update 12. The loss curve falls from 1800 at update 0 to nearly 0 by update 12.
- Unlike the stochastic path, the mean motion is **smooth** with no jitter, since it uses the average of the data instead of a random observation at each step. In this case, the **estimate goes down and the loss goes down** together.

Section 21: The Estimate Increases While the Loss Decreases

- The same data set and the same learning rate, $\gamma = 25\%$, can be used with only the **initial condition** changed. The **initial condition** is the starting value of the estimate. This time the estimate starts far below the average.

$$x[0] = -100$$

- At time 0, the discrepancy is negative, but squaring makes the loss positive and very large.

$$L[0] = \tfrac{1}{2}(-100 - 80)^2 = \tfrac{1}{2}(-180)^2 = \tfrac{1}{2}(32400) = 16200$$

- On the first update, subtracting 25% of a negative discrepancy moves the estimate upward.

$$x[1] = -100 - 0.25(-100 - 80) = -100 - 0.25(-180) = -100 + 45 = -55$$

$$L[1] = \tfrac{1}{2}(-55 - 80)^2 = \tfrac{1}{2}(-135)^2 = \tfrac{1}{2}(18225) = 9112.5$$

- On the second update, the estimate rises again.

$$x[2] = -55 - 0.25(-55 - 80) = -55 - 0.25(-135) = -55 + 33.75 = -21.25$$

$$L[2] = \tfrac{1}{2}(-21.25 - 80)^2 = \tfrac{1}{2}(-101.25)^2 = \tfrac{1}{2}(10251.5625) \approx 5125.8$$

- **The estimate rises toward 80**, climbing from -100 to about 74 by update 12. **The discrepancy shrinks** in size at every step, and so **one half of the discrepancy squared falls toward 0**, dropping from 16,200 at update 0 to nearly 0 by update 12.
- **The direction changes, but the learning does not.** Starting above the average, the estimate moves down; starting below the average, the estimate moves up. In both cases the loss decreases, which means that learning is defined by the loss going down, not by the estimate moving in any particular direction.

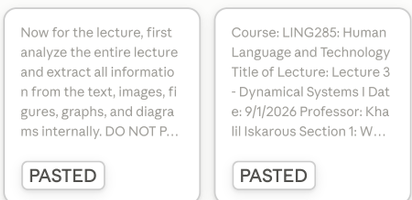
Section 22: Learning Rate and Initial Conditions

- SGD can be run for **300 epochs**, meaning 300 update steps, with three different initial conditions and four different learning rates. The three initial conditions are **IC = 130**, which starts well above the average; **IC = 80**, which starts right at the average; and **IC = 10**, which starts well below it. Every run settles around 80, the average of the data.

- With a **learning rate of 10%**, the run starting at 130 falls and the run starting at 10 rises, and both reach the neighborhood of 80 within about the first 25 to 50 epochs. After that, all three runs wiggle gently within a few units of 80, roughly between 75 and 85. The run starting at 80 stays near 80 throughout, with only small jitters.
- With a **learning rate of 50%**, the runs starting at 130 and at 10 reach the neighborhood of 80 within the first few epochs. After that, however, all three runs jump up and down widely, roughly between 70 and 95, and never settle down.
- With a **learning rate of 1%**, the runs move very slowly and very smoothly. After 300 epochs, the run starting at 130 has only come down to about 82, and the run starting at 10 has only climbed to about 77, so neither has fully reached the average. The run starting at 80 stays almost perfectly flat at 80 with very little jitter.
- With a **learning rate of 80%**, the runs starting at 130 and at 10 reach the average region almost immediately. After that, all three runs jump violently, roughly between 68 and 98, which is nearly the full range of the data values. With such a high learning rate, each update moves the estimate most of the way to whatever observation was just drawn, so the estimate mostly tracks individual data points instead of their average.
- The initial condition affects only the first part of each run. After enough epochs, all three runs behave the same way, and the size of the learning rate determines how fast they arrive and how much they jitter once they get there.

Section 23: The Fundamental Learning Tradeoff

- The learning rate creates a **fundamental learning tradeoff** between speed and accuracy.
  - When γ **increases**, learning **speed increases**, since each correction is larger and the estimate reaches the average region in fewer steps. At the same time, **accuracy decreases**, since the large corrections make the estimate jump around the average instead of settling on it.
  - When γ **decreases**, learning **speed decreases**, since each correction is small and the estimate needs many steps to reach the average. At the same time, **accuracy increases**, since the small corrections keep the estimate steady and close to the true average once it arrives.
- This tradeoff is an unhappy fact of learning: with a single learning rate, we cannot have the fastest learning and the most accurate result at the same time. The 1% run is accurate but slow, and the 50% and 80% runs are fast but noisy.
- **Thousands of people, for generations, have made their living** finding better ways to **optimize**, meaning to adjust a learning process so that it works as well as possible. The goal of this work is to find methods where **speed increases and accuracy increases** together, so that systems like LLMs can learn quickly while still ending up with accurate weights.



Course: LING285: Human Language and Technology
Title of Lecture: Lecture 3 - Dynamical Systems I
Date: 9/1/2026
Professor: Khalil Iskarous

Section 1: What a Dynamical System Is

- A **dynamical system** is a system whose parts change over time in coordinated ways. Dynamical systems appear in **animal behavior** and in **AI learning**.
- Every dynamical system is described by three parts: **1) Elements, 2) Relations, 3) Goal**. The **elements** are the pieces that make up the system, the **relations** are how those pieces are coordinated with each other, and the **goal** is what the coordinated system works to achieve.

Section 2: The Octopus as a Dynamical System

- An octopus defending its home is a dynamical system. Its **elements** are its **8 arms** and the **ground**.
- Its **relations** are the coordinated connections between each individual arm, the ground, and the fish. These relations are **planned by a neural system** that resides in the octopus's **head and the rest of its body**, so the planning is not located in the head alone.
- Its **goal** is to **keep its den**, meaning to protect its home from the fish.

Section 3: The Speech System as a Dynamical System

- Human speech is a dynamical system with many more elements. The **elements** are the **many parts of the tongue**, the **lips**, the **velum**, the **larynx**, the **palate**, and the **pharyngeal wall**.
    - The **velum** is the soft back part of the roof of the mouth, the **palate** is the hard roof of the mouth, the **larynx** is the voice box in the throat, and the **pharyngeal wall** is the back wall of the throat.
- The **relations** between all of these parts are **planned by the brain**, and the **goal** is to **convey a thought**.
- A **real-time MRI** recording of a speaker gives a side view of the head cut down the middle, with the lips at the front, the tongue filling the mouth, the palate and velum above it, and the throat and spine behind it. During about three seconds of speech, the lips, jaw, and tongue change shape and position continuously, so speech is many parts moving together over time.

Section 4: Dynamical Systems in You

- All of our **daily voluntary actions** are dynamical systems, and so are the **involuntary actions** of the **heart**, **pancreas**, and **neurons**.
- The elements of a dynamical system can be as large as **organs** or parts of organs, or as small as **cells** and the **chemicals** inside them. **Biology, chemistry, and physics** are all about dynamical systems, and the **speech system** is a **very complex dynamical system**.
- There are so many dynamical systems that we can **use one dynamical system to study another.** Two systems can be extremely different and still share an **underlying similarity** in how their elements, relations, and goals are organized.
    - This is described as **going up Diotima's ladder**, the idea from Plato's *Symposium* of rising from individual examples toward the general form they share. Studying many different dynamical systems moves us from single cases toward shared principles.

Section 5: The Octopus Arm "Slap" Study

- The octopus example comes from **"A syntactic analysis of a complex motor action: the octopus arm 'slap'"** by **Jean Alupay, Jennifer Mather, and Khalil Iskarous**, published in **Marine Biology (2023)**. The paper was received on 15 February 2023 and accepted on 1 June 2023.
- The researchers analyzed an octopus, ***Abdopus* sp.**, filmed in **tidal pools in Okinawa, Japan**, as it **slapped at fish** in the wild. In total, **19 slaps** were analyzed, and arm movement was measured in both an **external** reference frame, relative to the surroundings, and an **animal-centered** reference frame, relative to the octopus's own body.
- The octopus built each slap from two **kinematic primitives**, which are basic building-block movements: **rotation** and **translation** of its **hydrostatic arms**. It combined these primitives **serially**, one after another, and **in parallel**, at the same time.
    - A **muscular hydrostat** is a boneless structure supported by its own muscles, which is why the octopus arm can bend almost anywhere.
- By combining primitives, the octopus stays **flexible** while controlling **only a few degrees of freedom**, where a **degree of freedom** is one independent way the system can move. The authors call this **flexible rigidity**.
- The slap supports **Flash and Hochner's embodied organization** view of motor behavior, as well as their idea that **motor primitives combine syntactically** to form a complex action. Calling the combination **syntactic** means small movement units are combined by

rules into a larger action, the way words combine into sentences, so the octopus arm is being used to study the structure of language.

- The octopus uses **sensory feedback** about the position of the moving fish together with **feed-forward motor primitives**. **Feedback** is incoming sensory information used to adjust the movement, and **feed-forward** control sends out a planned movement without waiting for that information. Together they let the octopus build complex actions at **dynamic equilibrium with the environment**, keeping its actions matched to a changing situation.
- These findings give a more realistic view of how a complex behavior lets an animal **coordinate with its environment**. The paper's keywords are **Octopus, Muscular hydrostat, Motor control,** and ***Abdopus***.
- The book ***Octopus: The Ocean's Intelligent Invertebrate — A Natural History*** is by **Jennifer A. Mather, Roland C. Anderson, and James B. Wood**.

Section 6: AI as a Model of Animal Psychological Skills

- **AI** is a **model of several animal psychological skills**: **knowing, recognizing, attending, acting,** and **learning**. These are abilities animals already have, and AI systems are built to imitate them.
- The **Cloze Procedure** comes from **Wilson L. Taylor's** article **"'Cloze Procedure': A New Tool for Measuring Readability,"** published in ***Journalism Quarterly*** in **Fall 1953**.
  - In a **cloze task**, words are removed from a text and readers fill in the blanks, which is the skill of predicting a word from its context.
  - The article was the first full statement of the method and its theory, which had been introduced briefly at a workshop at the **1953 AEJ convention**. It reported **three pilot studies** and **two experiments** comparing cloze results with **two readability formulas**.

Section 7: How GPT Predicts the Next Word

- In **GPT**, each word is a **vector** containing **thousands of numbers**, and these vectors are called **x's** or **tokens**. In the input "**chickens cackle and ducks**," each of the four words is its own tall column of numbers.
- All of the AI's **knowledge** about speech, language, or anything else is stored in **a few trillion numbers** called **weights**, or **w's**.
- The AI combines its knowledge, the **W's**, with the input vectors, the **x's**, using mostly **addition, subtraction, multiplication, and division** ($+, -, \times, \div$).
- These calculations produce a **probability distribution function** over possible next words, which is the system's statement of its guess for the next word. A **probability distribution** gives each possible word a probability.
  - For "chickens cackle and ducks," the distribution is spread over possible words such as "**a**," "**cat**," "**quack**," and "**Zulu**." Many words receive small amounts of probability, and the tallest peak is over "**quack**," the system's best guess.
- How the **W's combine with the X's** is itself a **dynamical system**. **Learning** is an easier dynamical system than this prediction process.

Section 8: Where the Weights Come From

- The weights come from a training process summed up as **"Fake it till you make it!"** Training follows these steps:
  1. The system **starts with random W's**, which are completely random numbers of any value.
  2. The system is given a sequence of words, "**chickens cackle and ducks**," and asked to **predict the next word** by combining its W's with the input vectors. With random weights, its distribution has its tallest peak over the wrong word, "**cat**," with low, uneven probability elsewhere.
  3. The correct next word is known to be "**quack**." The **target distribution** is a single sharp spike on "quack" with nothing on any other word, and the system is told to **change its W's** so its output distribution looks more like this target.
  4. The system is exposed to **more sequences** and predicts the next word **billions of times**, changing its W's each time to **improve its guessing**.

- This cycle of guessing, comparing to the correct answer, and adjusting the weights is **how LLMs learn and know**.

Section 9: Two Tricks of AI Learning

- AI learning relies on **two tricks**, shown in two images generated by **ChatGPT**.
- The first trick is **"Fake It Till You Make It."** The system starts with random weights and makes predictions immediately, gaining knowledge through the process of predicting.
- The second trick is **"Learn by Making Mistakes."** Each wrong prediction gives an **error**, and the system uses that error to change its weights and improve later predictions.

Section 10: Averaging as Learning

- Averaging a short list of numbers is easy. The average of 10, 12, and 14 is their sum divided by how many there are.

$$\frac{10 + 12 + 14}{3} = 12$$

- This simple method **does not work for complex things** like **speech, language, biological molecules, and scenes**, which are the things AI works with. The averaging method used instead improves a guess step by step:
  - We start with a **list of numbers** and make a **wild guess** x about their average.
  - We improve x by **exposure to the data**. We take one number from the list and examine it with respect to the current x.
  - We move x **a bit closer** to that observed number. This is **observation-based error learning**, since each observation reveals an error and the guess is corrected toward the observation.
  - We **repeat** with another number.

Section 11: SGD Starts With a Wild Guess

- **SGD, stochastic gradient descent**, finds the average, the **essence**, of a list of numbers by learning from one observation at a time. The data set is the following list, and its true average is 80.25.

$$[78,\ 67,\ 98,\ 85,\ 76,\ 68,\ 98,\ 72]$$

$$\frac{78 + 67 + 98 + 85 + 76 + 68 + 98 + 72}{8} = \frac{642}{8} = 80.25$$

- SGD starts by picking **any random number** x as the estimate. At time 0 the estimate is 130, far above every number in the list, and the **learning rate** γ, the fraction of the error corrected on each step, is 25%.

$$x[0] = 130, \qquad \gamma = 25\% = 0.25$$

- The number in brackets is the time step, so $x[1]$ is the estimate after one update and $x[2]$ is the estimate after two updates.

Section 12: One Observation Creates Error

- A number d is **randomly picked** from the data. The first observation is 98, and the **discrepancy**, also called the **loss**, is the current estimate minus the observation.

$$d[0] = 98, \qquad 130 - 98 = 32$$

- The discrepancy of 32 means the estimate is 32 too high compared to this observation. The average should be **close to the data**, so this error signals that the estimate must change.

Section 13: Move 25% of the Error

- The estimate is corrected by **subtracting 25% of the discrepancy**.

$$x[1] = 130 - 0.25(32) = 130 - 8 = 122$$

- The estimate **moves toward the observation** but only one quarter of the way, from 130 to 122, so a single observation does not take over the estimate.

Section 14: Repeating the Correction

- Each update repeats the same procedure: draw a random observation, compute the discrepancy, and subtract 25% of it.
- On the second update, the observation is 85. The estimate moves toward it, from 122 to 112.75.

$$d[1] = 85, \qquad 122 - 85 = 37$$

$$x[2] = 122 - 0.25(37) = 122 - 9.25 = 112.75$$

- On the third update, the observation is 76. The estimate moves toward it, from 112.75 to about 103.563.

$$d[2] = 76, \qquad 112.75 - 76 = 36.75$$

$$x[3] = 112.75 - 0.25(36.75) = 112.75 - 9.1875 \approx 103.563$$

- On the fourth update, the observation is 68. The estimate moves toward it, from 103.563 to 94.672.

$$d[3] = 68, \qquad 103.563 - 68 = 35.563$$

$$x[4] = 103.563 - 0.25(35.563) = 103.563 - 8.891 = 94.672$$

- Over the first four updates, every observation was below the estimate, so the estimate fell steadily from 130 toward the average line at 80.25.

Section 15: A High Datum Pulls x Upward

- On the fifth update, the observation is 98, which is **above** the estimate. The discrepancy is therefore **negative**.

$$d[4] = 98, \qquad 94.672 - 98 = -3.328$$

$$x[5] = 94.672 - 0.25(-3.328) = 94.672 + 0.832 = 95.504$$

- **Subtracting a negative discrepancy moves x upward**, here from 94.672 to 95.504. The same rule works in both directions: when the estimate is above an observation it goes down, and when it is below an observation it goes up, so the estimate always moves toward the observation.

Section 16: Continued Corrections

- On the sixth update, the observation is 72, and the estimate drops from 95.504 to 89.628.

$$d[5] = 72, \qquad 95.504 - 72 = 23.504$$

$$x[6] = 95.504 - 0.25(23.504) = 95.504 - 5.876 = 89.628$$

- On the seventh update, the observation is 78, and the estimate drops from 89.628 to 86.721. The discrepancy is smaller than before, so the correction is smaller.

$$d[6] = 78, \qquad 89.628 - 78 = 11.628$$

$$x[7] = 89.628 - 0.25(11.628) = 89.628 - 2.907 = 86.721$$

- On the eighth update, the observation is 67, and the estimate drops from 86.721 to 81.791, only about 1.5 above the true average.

$$d[7] = 67, \qquad 86.721 - 67 = 19.721$$

$$x[8] = 86.721 - 0.25(19.721) = 86.721 - 4.930 = 81.791$$

Section 17: A High Datum Pulls x Upward Again

- On the ninth update, the observation is 98 again. Draws are random, so the same value can be picked more than once.

$$d[8] = 98, \qquad 81.791 - 98 = -16.209$$

$$x[9] = 81.791 - 0.25(-16.209) = 81.791 + 4.052 = 85.843$$

- The negative discrepancy moves x upward, from 81.791 to 85.843. A single high observation pushed the estimate away from the true average, since each step responds only to the one observation drawn.

Section 18: Final Corrections

- On the tenth update, the observation is 85, very close to the estimate. The discrepancy is less than 1, so the estimate barely changes, from 85.843 to 85.632.

$$d[9] = 85, \qquad 85.843 - 85 = 0.843$$

$$x[10] = 85.843 - 0.25(0.843) = 85.843 - 0.211 = 85.632$$

- On the eleventh update, the observation is 76, and the estimate drops from 85.632 to 83.224.

$$d[10] = 76, \qquad 85.632 - 76 = 9.632$$

$$x[11] = 85.632 - 0.25(9.632) = 85.632 - 2.408 = 83.224$$

- On the twelfth update, the observation is 68, and the estimate drops from 83.224 to 79.418, less than 1 below the true average of 80.25.

$$d[11] = 68, \qquad 83.224 - 68 = 15.224$$

$$x[12] = 83.224 - 0.25(15.224) = 83.224 - 3.806 = 79.418$$

- Across twelve updates, the estimate went 130, 122, 112.75, 103.563, 94.672, 95.504, 89.628, 86.721, 81.791, 85.843, 85.632, 83.224, and 79.418. It fell steeply at first, rose twice when 98 was drawn, and ended near the average line.

Section 19: That Is SGD

- **SGD** has two steps. First, **make an initial estimate or guess**. Second, **use data to improve the estimate**.
- Each word in the name describes part of the method.
  - **Stochastic** means **random observation**: each correction uses a data point picked at random.
  - **Gradient** means a **loss-related direction**: the discrepancy tells the estimate which way to move, down when positive and up when negative.
  - **Descent** means **reducing the mistake**.
- **The path jitters, but its tendency is toward the average.** Individual steps can move away from the average, but repeated observations move x toward about 80.

Section 20: As the Estimate Decreases, the Loss Decreases

- A **second data set** has an average of exactly 80.

$$[62,\ 70,\ 78,\ 82,\ 90,\ 98]$$

$$\frac{62 + 70 + 78 + 82 + 90 + 98}{6} = \frac{480}{6} = 80$$

- The **mean motion of SGD** is how the estimate moves on average, which is toward the mean of the data. It follows this rule, where t is any time step.

$$x[t+1] = x[t] - 0.25\,(x[t] - 80)$$

- The **loss** is one half of the discrepancy squared. **Squaring** makes the loss positive whether the estimate is too high or too low, and the loss equals zero only when the estimate equals 80.

$$L[t] = \tfrac{1}{2}\,(x[t] - 80)^2$$

- Starting at $x[0] = 140$, the first two updates are worked out below.

$$L[0] = \tfrac{1}{2}(140 - 80)^2 = \tfrac{1}{2}(3600) = 1800$$

$$x[1] = 140 - 0.25(60) = 140 - 15 = 125, \qquad L[1] = \tfrac{1}{2}(45)^2 = \tfrac{1}{2}(2025) = 1012.5$$

$$x[2] = 125 - 0.25(45) = 125 - 11.25 = 113.75, \qquad L[2] = \tfrac{1}{2}(33.75)^2 \approx 569.5$$

- Each update removes 25% of the discrepancy, so the discrepancy shrinks from 60 to 45 to 33.75. Over 12 updates, the estimate falls quickly and then levels off near 80, reaching about 82, while the loss falls from 1800 to nearly 0. The mean motion is **smooth**, without the jitter of random observations, and here **the estimate decreases and the loss decreases** together.

Section 21: The Estimate Increases While the Loss Decreases

- With the **same data** and the **same** $\gamma = 25\%$, only the **initial condition**, the starting value of the estimate, is changed to -100.

$$L[0] = \tfrac{1}{2}(-100 - 80)^2 = \tfrac{1}{2}(-180)^2 = \tfrac{1}{2}(32400) = 16200$$

$$x[1] = -100 - 0.25(-180) = -100 + 45 = -55, \qquad L[1] = \tfrac{1}{2}(-135)^2 = \tfrac{1}{2}(18225) = 9112.5$$

$$x[2] = -55 - 0.25(-135) = -55 + 33.75 = -21.25, \qquad L[2] = \tfrac{1}{2}(-101.25)^2 \approx 5125.8$$

- **The estimate rises toward 80**, reaching about 74 by update 12. **The discrepancy shrinks**, so **one half of the discrepancy squared falls toward 0**, from 16,200 to nearly 0.
- **Direction changes; learning does not.** Starting above 80, the estimate falls, and starting below 80, it rises, but in both cases the loss decreases. Learning is the loss going down, not the estimate moving in a particular direction.

Section 22: Learning Rates and Initial Conditions

- SGD is run for **300 epochs** (update steps) from three initial conditions, **IC = 130**, **IC = 80**, and **IC = 10**, at four learning rates. All runs end up around 80.
- At a **learning rate of 10%**, the runs from 130 and from 10 reach about 80 within roughly 25 to 50 epochs. After that, all three runs wiggle gently within a few units of 80, about 75 to 85, and the run from 80 stays near 80 throughout.
- At a **learning rate of 50%**, the runs from 130 and from 10 reach the 80 region within the first few epochs. After that, all three runs jump widely, roughly between 70 and 95.
- At a **learning rate of 1%**, the runs move slowly and smoothly. After 300 epochs, the run from 130 has only fallen to about 82 and the run from 10 has only risen to about 77, while the run from 80 stays almost flat at 80.
- At a **learning rate of 80%**, the runs from 130 and from 10 reach the 80 region almost immediately. After that, all three runs jump violently between roughly 68 and 98, nearly the full range of the data, since each update moves the estimate most of the way to the observation just drawn.
- The initial condition affects only the start of each run. The learning rate determines how fast the estimate arrives near the average and how much it jitters afterward.

Section 23: Fundamental Learning Tradeoff

- The learning rate creates a **fundamental tradeoff** between **speed** and **accuracy**.
  - When γ **increases**, **speed increases**, since each correction is larger and the estimate reaches the average in fewer steps. **Accuracy decreases**, since the large corrections make the estimate jump around the average.

    - When γ **decreases**, **speed decreases**, since each correction is small and many steps are needed. **Accuracy increases**, since the small corrections keep the estimate steady near the average.
- A single learning rate cannot give both the fastest learning and the most accurate result, which is an unhappy fact of learning.
- **Thousands of people for generations** have made their living finding better ways to **optimize**, which means adjusting the learning process so that **speed increases** and **accuracy increases** at the same time.